User Story and Summary

This API allows games to define, track, and manage achievements for users. It allows for games to register new achievements, record when users unlock them, and update progress for achievements that require a count.

Our user story is: As a player, I would like to track and earn achievements for the games I play.

API Documentation

Note: Previously all endpoints (besides /achievements/add) expected a `userId` int. This has been updated to a `username` string.

For username to be utilised accordingly the player must be registered within the player hub/player interact feature. The player must be prompted within the game for their username which must match the username used for this feature. Otherwise our API can not store information against this user. We encourage teams to interact with the player hub feature created by Coding Axolotls to get the maximum benefit/use of this API.

The Achievements API provides the following methods for games to interact with;

1. Register a New Achievement

- HTTP Method: 'POST'
- Endpoint: '/achievements/add'
- Purpose: Creates and registers a new achievement type within the system.
- Request Body (JSON): //id will act as is unique achievement identifier (eg. first_win1)

```
{
    "id": "string",
    "gameName": "string",
    "title": "string",
    "description": "string"
}
```

2. Unlock an Achievement for a User

- HTTP Method: 'POST'
- Endpoint: '/achievements/unlock'
- Purpose: Records that a specific achievement has been unlocked for a given user.
- Request Body (JSON): //achievementId is id defined when the achievement was added

```
{
   "username": "string",
   "achievementId": "string"
}
```

3. Update Achievement Goal for a User

- HTTP Method: 'PUT'
- Endpoint: '/achievements/goal'
- Purpose: Updates the target goal for a specific achievement for a given user. If a progress entry doesn't exist, it creates one (initialising count to 0).
- Request Body (JSON):

```
{
   "username": "string",
   "achievementId": "string",
   "targetGoal": 10
}
```

4. Update Achievement Progress for a User

- HTTP Method: 'PUT'
- Endpoint: '/achievements/progress'
- Purpose: Updates a user's progress towards a goal-based achievement
- Request Body (JSON): // the user id is the ID of the user whose progress is being updated, integer value from users table

```
{
   "username": "string",
   "achievementId": "string",
   "count": 5
}
```

5. Get all unlocked achievements for a user

- HTTP Method: 'GET'
- Endpoint: '/achievements/:username'
- Purpose: Retrieves a list of all achievements that a specific user has unlocked.
- Response Body (JSON):

```
[
    {
        "username": "string",
        "achievementId": "achievement_id_1",
        "unlockedAt": "Timestamp"
    },
    {
        "username": "string",
        "achievementId": "achievement_id_2",
        "unlockedAt": "Timestamp"
    }
]
```

5. Get specific achievement progress for a user

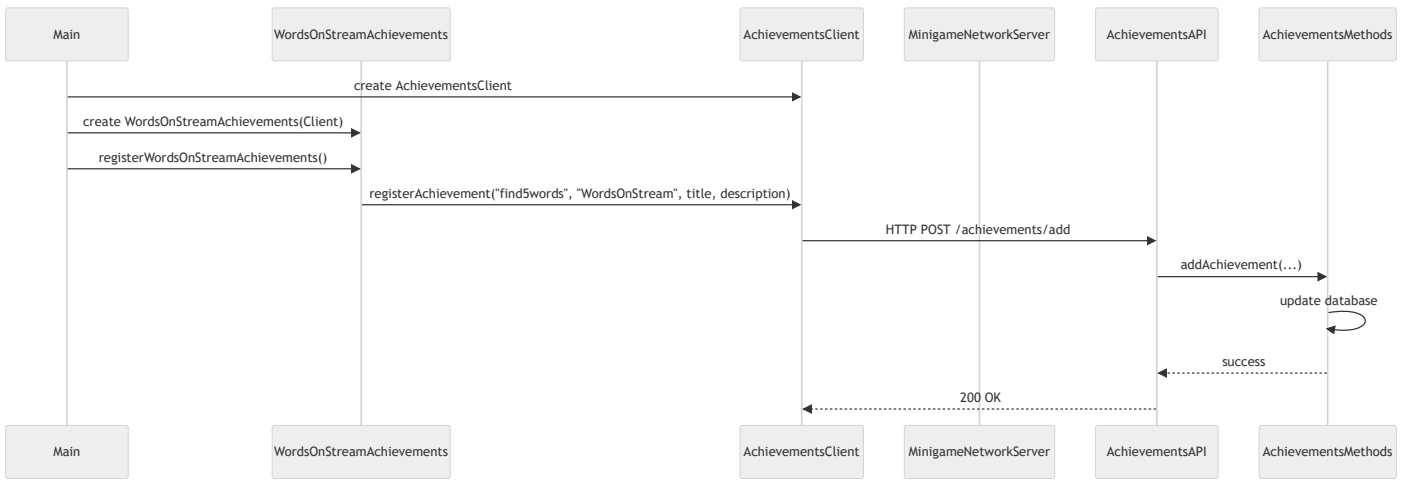
- HTTP Method: 'GET'
- Endpoint: /achievements/progress/:username/:achievementId
- Purpose: Retrieves a list of all achievements that a specific user has unlocked.
- Response Body (JSON): //will update formatting for time

```
{
    "username": "string",
    "achievementId": "achievement_id_x",
    "count": 5,
    "goal": 10
}
```

Database schema

Please see the image below for the database schema. Please note the users table is developed by the coding axolotls and will be a key reference for the tables we have added in our schema.

Register (Add) Achievement to Database Sequence Diagram



How to add achievements to a game

See here.

Class Documentation

AchievementsAPI

- Location: server/AchievementsClient.java
- Contains API endpoints for sending and retrieving achievement data from the database.
- Uses methods defined in `AchievementsMethods` to interact with the database.

Method	Purpose

Method	Purpose
`registerRoutes`	Registers all achievement-related API routes (`/achievements/add` , `/achievements/unlock` , `/achievements/progress` , `/achievements/unlocked/:username` , `/achievements/progress/:username/:achievementId`) with the Vert.x router.
`addAchievement`	Handles `POST /achievements/add` . Accepts a JSON body mapping to `AchievementsDTO` (`id` , `gameName` , `title` , `description`) and adds a new achievement record via `achievementsMethods.addAchievement`
`unlockAchievement`	Handles `POST /achievements/unlock` . Accepts a JSON body mapping to `UnlockAchievementDTO` (`username` , `achievementId`) and marks the achievement as unlocked for the user
`updateProgress`	Handles `PUT /achievements/progress` . Accepts a JSON body mapping to `AchievementsProgressDTO` (`username` , `achievementId` , `count`) and updates progress toward unlocking that achievement
`getUnlockedAchievements`	Handles `GET /achievements/unlocked/:username` . Retrieves all unlocked achievements for the given `userId` and returns them as JSON on success. Returns 400 if `userId` is invalid.
`getAchievementProgress`	Handles `GET /achievements/progress/:username/:achievementId` . Retrieves progress data for a specific achievement for the given user and returns it as JSON on success. Returns 400 if `userId` is invalid.

WordsOnStreamAchievements

- Location: server/wordsonstream/WordsOnStreamAchievements.java
- Helper class that wraps AchievementsClient to register and manage achievements specific to the Words On Stream game

Method	Purpose
`registerWordsOnStreamAchievements`	Iterates over the `availableAchievements` map (ID → [title, description]) and calls `AchievementsClient.registerAchievement` for each one to register all Words On Stream achievements with the backend.
`unlockFind5Words`	Calls `client.unlockAchievement(username, "find5words")` to mark the "Find 5 words" achievement as unlocked for a given user.
`unlockReachLevel3`	Unlocks the "Reach Level 3" achievement for a given user.
`unlockLongWord`	Unlocks the "Long Word" achievement for a given user.
`updateConnectFour`	Updates progress toward the "Connect Four" achievement by sending the current count for a given user.

AchievementsClient

- Provides a simple client for other server components to interact with the `AchievementsAPI` endpoints via HTTP requests using the Vert.x `WebClient` .

Method	Purpose
`registerAchievement`	Sends a `POST /achievements/add` request with JSON `{id, gameName, title, description}` to register a new achievement in the database.
`unlockAchievement`	Sends a `POST /achievements/unlock` request with JSON `{username, achievementId}` to unlock an achievement for the given user.
`updateProgress`	Sends a `PUT /achievements/progress` request with JSON `{username, achievementId, count}` to update progress toward an achievement for the given user.

Comments